Disciplina: Inteligência Artificial – BCC IBILCE/UNESP
2026

Prof. Dr. Wallace Casaca ⟨`wallace.casaca@unesp.br`⟩
Monitor: Lucas Alves Sanches ⟨`alves.sanches@unesp.br`⟩

Trabalho 1

O Trabalho 1 contém questões teóricas cujas respostas devem ser entregues, via Google Classroom, juntamente com os códigos da resolução prática de cada questão (as respostas podem ser apresentadas em um único arquivo Notebook).

Enunciado

O quebra-cabeça de 8 peças consiste em um tabuleiro 3×3 com oito peças numeradas e um quadrado vazio. O objetivo é alcançar um estado objetivo específico deslizando as peças horizontal ou verticalmente para o espaço vazio (Figura 1).

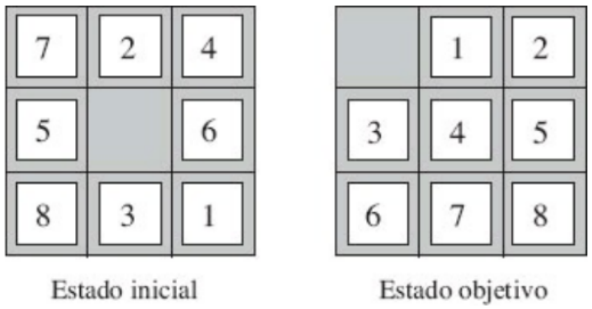

FIGURA 1. Exemplo típico de instância do quebra-cabeças de oito peças.

Embora o espaço de estados desse problema seja gerenciável (existe um número finito de estados acessíveis), as versões maiores como o de 15 peças tornam a busca cega inviável. Para resolver esse problema de forma eficiente, utilizamos a busca informada (heurística), que usa conhecimentos específicos do problema para guiar o algoritmo até a solução de maneira mais rápida do que uma busca cega.

Nesta atividade, deve ser implementado o algoritmo A* em Python para encontrar a sequência ótima de movimentos que leva de um estado inicial aleatório ao estado objetivo. A solução de menor custo $f(n)$ é apresentada abaixo.

$$f(n) = g(n) + h(n), \tag{1}$$

onde $g(n)$ é o custo do caminho desde o nó inicial até o nó n e $h(n)$ é a estimativa heurística até o objetivo.

Para esta atividade, o código deverá implementar duas heurísticas para uma posterior análise.

- h_1 = número de peças fora de lugar. Para a Figura 1, todas as oito peças estão fora de lugar, de modo que o estado inicial teria $h_1 = 8$.
- h_2 = soma das distâncias das peças de suas posições-objetivo (apenas a soma dos movimentos horizontais e verticais), também conhecida como distância de Manhattan. Dessa forma, as peças de 1 a 8, na Figura 1, resultariam em uma distância de Manhattan de $h_2 = 3 + 1 + 2 + 2 + 2 + 3 + 3 + 2 = 18$.

Além disso, para que o algoritmo seja eficiente, utilize o módulo `heapq` para gerenciamento da borda (fila de prioridade). Isso garante que o nó com o menor valor de $f(n)$ seja sempre o primeiro a ser extraído. Também utilize um `set` ou `dict` para armazenar estados já visitados, permitindo verificar se um estado é repetido. Dessa forma, você utilizará o `heapq` para manter os nós ordenados por $f(n)$ (borda), usará um `set` para guardar as tuplas que representam o tabuleiro (explorados) e para a busca, o seu laço de iteração pegará o nó com menor $f(n)$ da `heapq`, verificará se é o objetivo, se não for, verificará se o estado já está no `set` de explorados e, se for novo, gerará os vizinhos e os colocará na `heapq` (Algorítimo 1).

```
função BUSCA-DE-CUSTO-UNIFORME(problema) retorna uma solução ou falha
    nó ← um nó com ESTADO = problema.ESTADO-INICIAL, CUSTO-DE-CAMINHO = 0
    borda ← fila de prioridade ordenada pelo CUSTO-DE-CAMINHO, com nó como elemento
único
    explorado ← um conjunto vazio
    repita
        se VAZIO?(borda), então retornar falha
        nó ← POP(borda) / * escolhe o nó de menor custo na borda */
        se problema.TESTE-OBJETIVO(nó.ESTADO) então retornar SOLUÇÃO(nó)
        adicionar (nó.ESTADO) para explorado
        para cada ação em problema. AÇÕES(nó.ESTADO) faça
            filho ← NÓ-FILHO (problema, nó, ação)
            se (filho.ESTADO) não está na borda ou explorado então
                borda ← INSIRA (filho, borda)
            senão se (filho.ESTADO) está na borda com o maior CUSTO-DE-CAMINHO então
                substituir aquele nó borda por filho
```

Algorítimo 1: Busca de custo uniforme em um grafo. Note que este algoritmo é idêntico ao algoritmo da busca A*, exceto que A* usa $g + h$ em vez de apenas g (CUSTO-DE-CAMINHO).

Para realizar os testes, você deve criar pelo menos 10 instâncias com níveis crescentes de dificuldade para cada heurística, ou seja, você deve aumentar a profundidade (d) da solução ótima (número mínimo de movimentos). Para isso, parta do estado objetivo e execute N movimentos aleatórios para embaralhar o tabuleiro (por exemplo: $N = 2, 4, 8, 16, 20, \ldots, 26$ passos). Além disso, para cada par heurística-profundidade, você deve executar no mínimo 100 problemas aleatórios diferentes, registrando o custo da busca (número de nós gerados) e o tempo de execução de cada um, calculando-se a média desses valores para aquele nível de profundidade. Dessa forma, você será capaz de obter o fator de ramificação efetivo (b^*) médio de cada profundidade.

O fator de ramificação efetivo caracteriza a qualidade de uma heurística. Se o número total de nós gerados por A* para um problema particular for N e a solução da profundidade for d, então b^* é o fator de ramificação que uma árvore uniforme de profundidade d teria de ter a fim de conter $N + 1$ nós. Assim,

$$N + 1 = 1 + b^* + (b^*)2 + ... + (b^*)d. \tag{2}$$

Após a implementação do código em Python, você deve gerar uma tabela de comparação de $A^*(h_1)$ e $A^*(h_2)$, apresentando o custo da busca (quantidade de nós gerados) e o fator de ramificação efetivo b^* de cada profundidade d. Em seguida, responda:

(1) Qual heurística apresentou o menor número de nós gerados e o menor fator de ramificação efetivo (b^*)?

(2) É correto afirmar que h_2 domina h_1? Como essa relação impacta matematicamente o número de nós expandidos pelo algoritmo A*?

(3) Por que ambas as heurísticas são consideradas admissíveis? O que aconteceria com a otimalidade do algoritmo se a heurística superestimasse o custo real?

(4) Qual é a principal limitação do A* ao lidar com problemas de maior escala (como o quebra-cabeça de 15 peças) e como o armazenamento dos nós impacta a memória?